PROJECT DETAIL • SOFTWARE SPECIFICATION

NutriSense

AI-Powered Nutrition, Diet & Clinical Health Companion

Technical Project Detail & Architecture Document

Prepared by **Jamal Matloob**

Version 1.0 | September 2026 | Confidential

Table of Contents

1. Executive Summary & Project Introduction

NutriSense is an enterprise-grade, AI-powered nutritional intelligence, fasting management, and clinical health platform designed specifically for Pakistani and South Asian users. Engineered to combat the national surge in lifestyle-related metabolic disorders—principally Type 2 Diabetes, Hypertension, and Obesity—NutriSense makes personalized, culturally-accurate nutrition and fitness guidance accessible to every household, regardless of income.

1.1 Purpose & Problem Statement

The Problem: Pakistan faces a nutrition crisis – over 33% of adults are diabetic or pre-diabetic, with rising obesity, micronutrient deficiencies, and a high poverty rate that leaves most families unable to afford professional nutritionist consultations. Existing nutrition apps are built for Western diets: they ignore traditional portion units (katori, roti, naan), don't account for fasting practices like Ramadan, and are English-only – excluding the majority of users. Patients with diabetes or hypertension receive no personalized, clinically-safe dietary or exercise guidance, and families managing nutrition for children or elderly parents have no unified tool.

1.2 Target Audience

Who it affects: Underweight individuals, diabetics, people with high blood pressure, and those with obesity – the groups most vulnerable to preventable nutrition-related health complications.

1.3 Key Value Propositions & Core Differentiators

The Solution: NutriSense is an AI-powered nutrition and metabolic health assistant designed specifically for Pakistani and South Asian users. Using Google Gemini AI, it calculates personalized macro targets based on medical conditions and recommends tailored workout and exercise strategies matched to each user's needs – such as strength training and calorie-dense meal plans for underweight individuals, and safe, low-impact routines for diabetics and hypertension patients. It provides a clinically-guarded AI coach that detects high-risk patterns (like hypoglycemia) and escalates to care. It includes a Ramadan fasting mode with Sehri/Iftar schedules and split hydration, multi-family profile management, bilingual English/Urdu support, offline-first sync, and weekly PDF health reports. NutriSense makes personalized, culturally-accurate nutrition and fitness guidance accessible to every Pakistani household – regardless of income.

2. System Architecture & Tech Stack Specification

NutriSense adopts a modern, decoupled, multi-tier client-server architecture designed for high availability, sub-second latency, and enterprise security compliance.

2.1 High-Level Layered Architecture Overview

The system is organized into five distinct architectural tiers:

1. **Presentation Layer (Client):** Multi-platform Flutter application providing responsive, high-contrast dark-mode UI with bilingual Urdu/English localization.
2. **State & Service Controller Layer:** Reactive ListenableBuilder controllers, Singleton ViewModels, and background sync workers managing local business logic.
3. **Backend API Gateway Layer:** Asynchronous FastAPI microservices providing secure REST endpoints, input sanitization, and structured JSON schemas.
4. **AI & Intelligence Layer:** Google Gemini LLMs governed by a multi-key failover pool and clinical safety guardrail engines.
5. **Data Persistence Layer:** Supabase PostgreSQL with Row Level Security (RLS) policies coupled with an offline-first mobile SQLite caching layer.

2.2 Frontend Architecture & Client-Side Stack

The mobile and web client is engineered using Flutter 3.x, prioritizing native performance, modularity, and accessibility.

Component Layer	Technology / Library	Architectural Purpose & Scope
Framework	Flutter 3.x / Dart 3.x	Cross-platform high-performance rendering engine (Android, iOS, Web).
State Management	Provider & ListenableBuilder	Lightweight, reactive state propagation for ViewModels and controllers.
Local Database	sqflite / SQLite	Offline-first transactional local cache for meal logs, water logs, and habits.
Preferences	shared_preferences	Secure local persistence for tokens, language, and Ramadan state flags.
Health Integration	health package (v12+)	Bi-directional bridge to Google Health Connect and Apple HealthKit.
Voice & Speech	flutter_tts	Bilingual Text-To-Speech audio synthesizer for English and Urdu coaching.
Networking	http & connectivity_plus	REST API communication with dynamic network status monitoring.

2.3 Backend Architecture & Server-Side Stack

The backend infrastructure is built on FastAPI (Python 3.11+ ASGI), chosen for native asynchronous I/O and rapid JSON serialization.

Service Module	Core Technology	Operational Capability & Responsibilities
API Server	FastAPI / Uvicorn (ASGI)	Asynchronous endpoints handling meal search, coaching, and sync.
Cloud Database	Supabase PostgreSQL	Relational database with strict Row Level Security (RLS) and Auth.
Authentication	Supabase Auth (JWT)	Encrypted JSON Web Tokens with bearer verification middleware.
PDF Reporting	ReportLab Engine	Generates high-resolution weekly clinical receipts and dietary summaries.
HTTP Client	httpx (Async)	High-throughput asynchronous communication with external third-party APIs.

2.4 AI Engine & Multi-Key Failover Pool Architecture

NutriSense leverages Google's flagship Gemini models (gemini-2.5-flash and gemini-3.6-flash). To ensure 99.9% uptime and prevent service denial from Google API rate limits, a custom GeminiPool manager executes automatic round-robin load distribution and instant failover.

> 📌 **Resilient AI Pool Design Pattern**
> When an inference call triggers an HTTP 429 Quota Exceeded error, GeminiPool catches the exception, masks sensitive credentials in logs, instantly rotates the active index to the next verified key, and transparently completes inference in <500ms without user interruption.

2.5 External Integrations & Third-Party APIs

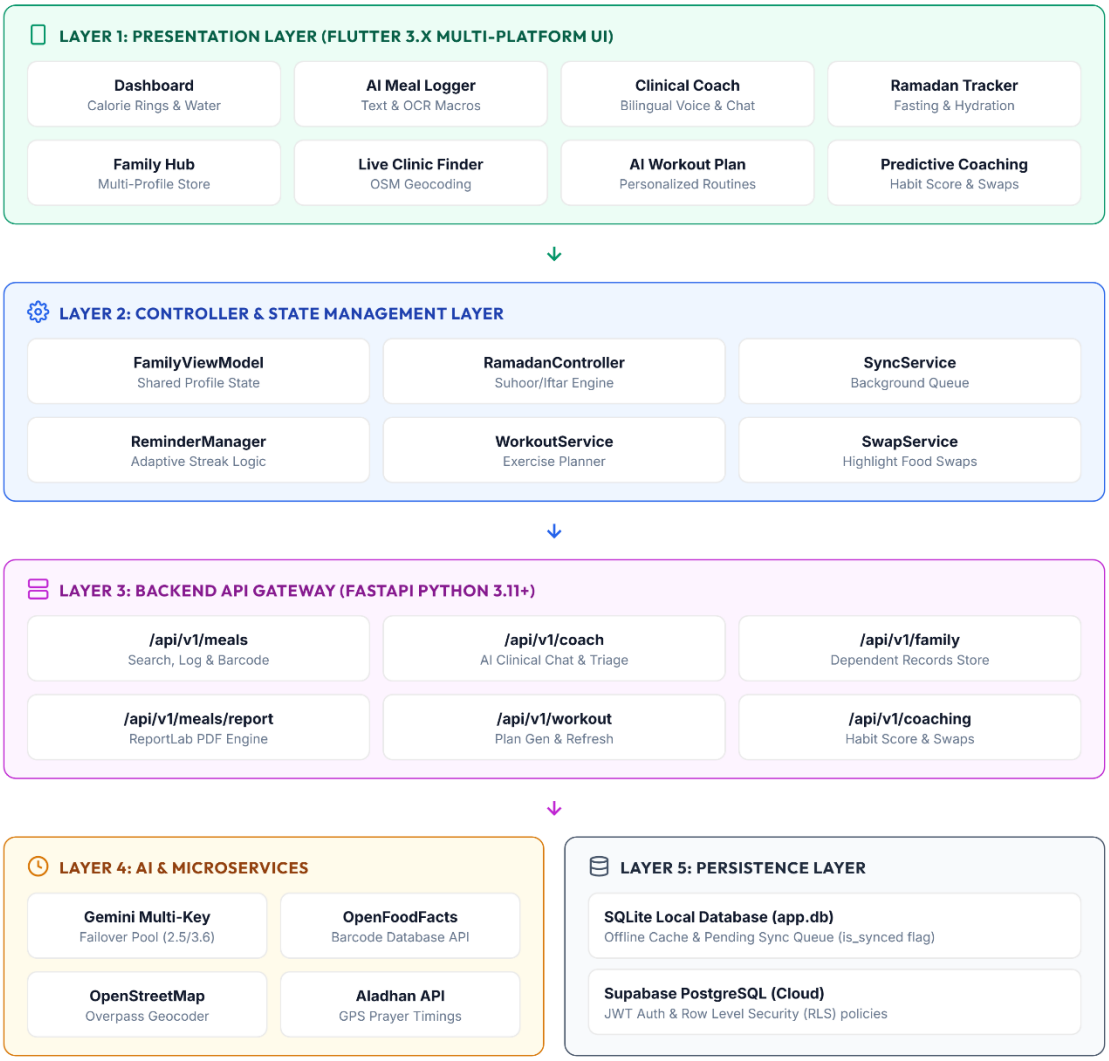
- **OpenFoodFacts API:** Global barcode lookup providing raw ingredient and nutritional metadata for packaged goods.
- **OpenStreetMap / Overpass API:** Geospatial query engine discovering certified emergency hospitals, clinics, and pharmacies within a 5km to 15km user radius.
- **Aladhan Prayer API:** Astronomical calculation engine providing exact daily Fajr (Sehri) and Maghrib (Iftar) timestamps based on GPS coordinates.

3. Software Engineering Diagrams & Specifications

Here are eight foundational architectural and behavioural diagrams specifying NutriSense's software design. Each specification outlines the actors, component interactions, lifecycle states, and data pathways.

3.1 High-Level System Architecture Diagram

Figure 01 : High-Level System Architecture Diagram

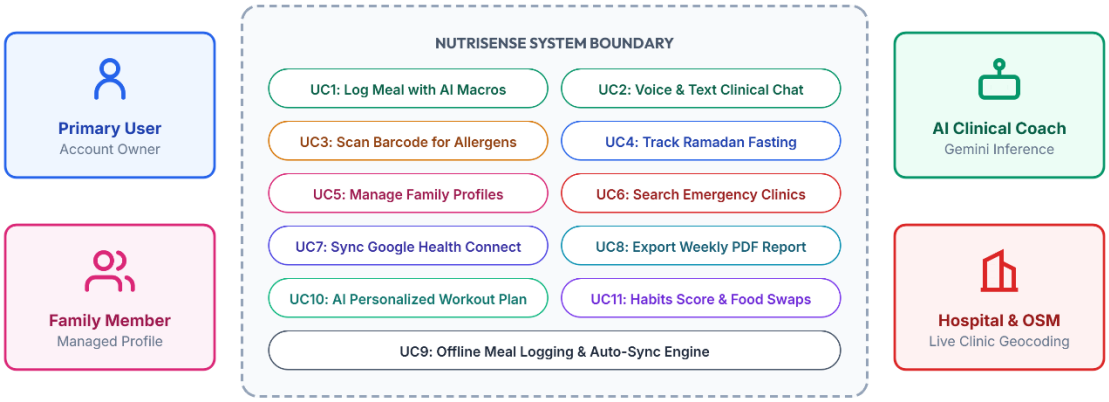


- **Tiered Decoupling:** The system is structured into 5 isolated tiers (Presentation, Controller, Gateway, AI/Microservices, and Persistence) to maintain clean separation of concerns.
- **Reactive Data Binding:** The Presentation layer (Flutter UI) communicates with the Controller layer using ViewModels and change notifier listeners, ensuring zero-latency interface updates.

- **Central API Routing:** The Controller tier sends queries through a network client to FastAPI server endpoints, which serve as the unified gatekeeper for business logic.
- **Asynchronous Services:** The Backend server coordinates external integrations (Gemini AI, OpenStreetMap, and OpenFoodFacts) asynchronously without blocking database transactions.
- **Dual-Database Persistence:** Write actions are committed locally to a SQLite mobile database and mirrored securely in a remote Supabase PostgreSQL cloud database.

3.2 Use Case Diagram

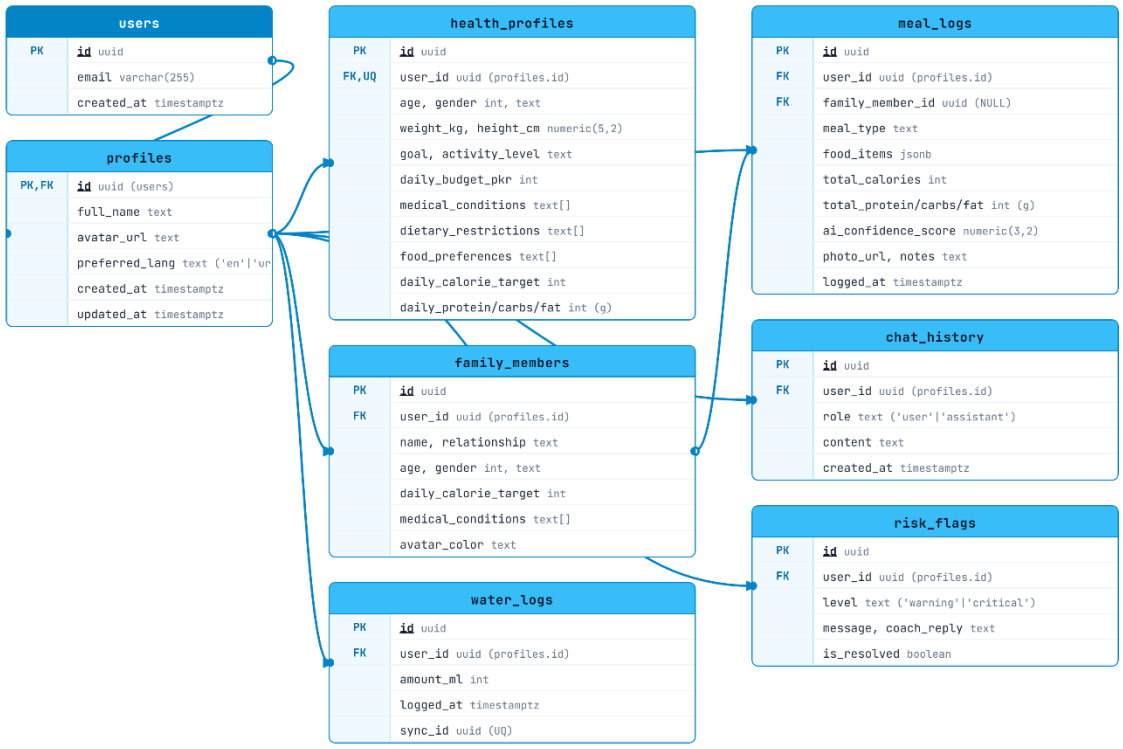
Figure 02 : Use Case Diagram



- **Actor Classifications:** The system separates human actors (Primary Users and Family Dependents) from automated system actors (AI Health Coach and Emergency Map Services).
- **Primary Workflows:** Primary Users can execute core use cases such as log meals, consult the AI Coach, enable fasting mode, and switch profiles to track children or parents.
- **AI Coach Interactions:** The AI Clinical Coach initiates active use cases like symptom triage, alert escalations, and Urdu/English text-to-speech voice guidance.
- **System Boundaries:** Integrations like Google Health Connect (for health sync) and OpenStreetMap (for clinic routing) are modeled outside the primary application boundary.
- **Offline Operations:** Use cases explicitly define the transition between offline-capable caching and background synchronizations when connectivity is restored.

3.3 Entity-Relationship Diagram

Figure 03 : Entity-Relationship Diagram



The database schema is normalized to 3NF, enforcing referential integrity and user isolation via foreign key cascades:

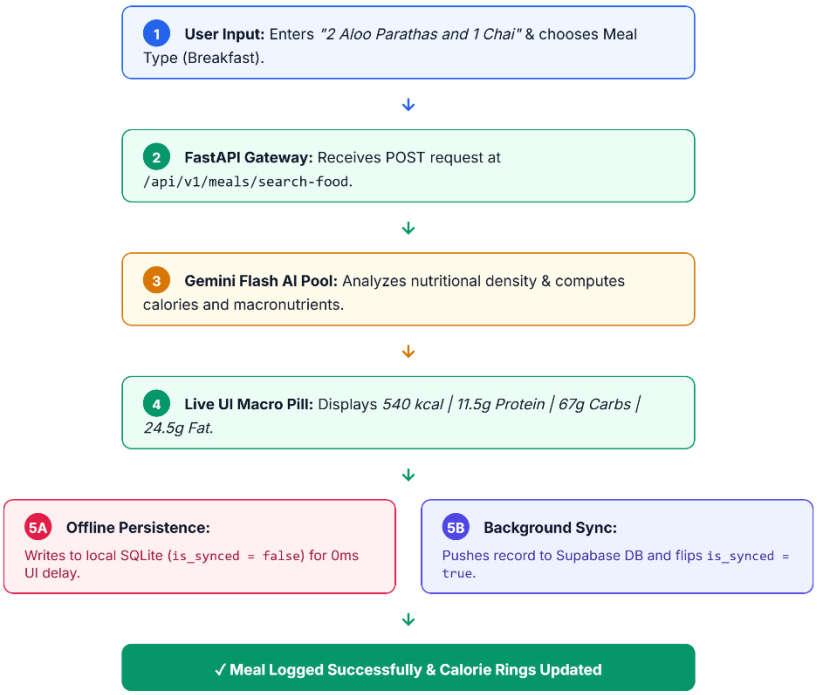
Table Name	Key Columns & Types	Relational Constraints & Description
users	id (UUID PK), email, created_at	Root Supabase Auth entity representing registered accounts.
health_profiles	user_id (UUID FK), age, height, weight, goal, conditions[], allergies[]	1:1 with users. Stores clinical metrics, daily targets, and allergies.
family_members	id (UUID PK), primary_user_id (FK), name, relationship, age, conditions[]	1:N with users. Stores dependent sub-profiles for family tracking.
meal_logs	id (UUID PK), user_id (FK), family_member_id (FK), notes, calories, macros	Stores logged meals with macro breakdown and offline sync flags.
chat_history	id (UUID PK), user_id (FK), sender, message, metadata, timestamp	Maintains multi-turn conversations between user and AI Health Coach.
water_logs	id (UUID PK), user_id (FK), amount_ml, logged_at	Logs hydration entries for daily water target tracking.

habit_scores	id (UUID PK), user_id (FK), score, streak_days, last_calculated	Maintains 30-day algorithmic consistency scores and streaks.

- **Strict Authorization:** The database schema anchors all tables to the core `users` authentication entity, securing data rows with PostgreSQL Row-Level Security (RLS).
- **1-to-1 Profile Mapping:** The `health_profiles` table maintains a 1-to-1 relationship with `users` to store clinical metrics, medical conditions, and allergy rules.
- **1-to-N Household Hierarchies:** The primary user's account links to a child `family_members` table to allow tracking dependents with isolated calorie and allergy thresholds.
- **Transactional Event Logs:** Tables for `meal_logs`, `water_logs`, and `chat_history` store discrete logged events with timestamps, foreign keys, and synchronization markers.
- **Algorithmic Score tracking:** The `habit_scores` table tracks rolling 30-day nutrition scores and tracking streak milestones to drive user behavior consistency.

3.4 AI Meal Logging & Macro Estimation Flowchart

Figure 04 : AI Meal Logging & Macro Estimation Flowchart



- **Conversational Prompt Capture:** The user enters a natural language meal text (e.g., '1 plate Daal Chawal with salad') which is sent directly to the FastAPI server.
- **Prompt Grounding:** FastAPI passes the query to the Gemini Flash model, grounded with South Asian food dictionaries, portion metrics, and cooking styles.
- **Structured JSON Estimation:** Gemini outputs a validated JSON payload containing the calorie count, portions, protein, fats, and carbs.

- **Instant Local Commit:** On user confirmation, the Flutter app instantly saves the entry in the local SQLite table (`is_synced = false`) to refresh UI rings instantly.
- **Background Data Mirror:** A background sync worker uploads the meal log to Supabase in a transaction and switches the SQLite sync flag to `true`.
- **Lifecycle Execution:**

(1) User inputs '1 plate Chicken Biryani with Raita'.

(2) Flutter sends JSON payload to FastAPI `/meals/search-food`.

(3) FastAPI prompts GeminiPool with USDA and South Asian culinary groundings.

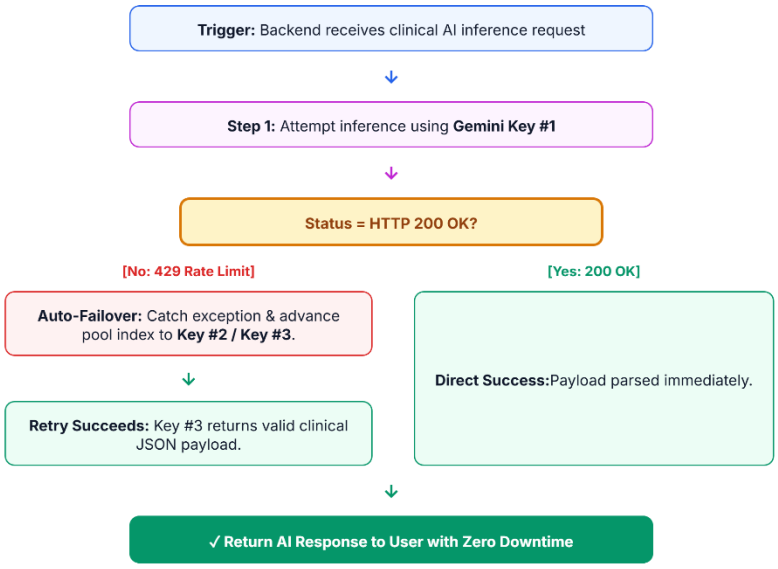
(4) Gemini returns structured JSON: 520 kcal, 28g protein, 65g carbs, 16g fat.

(5) UI displays macro summary for 1-tap confirmation.

(6) On confirmation, row is written to local SQLite and queued for background Supabase push.

3.5 Gemini Multi-Key Failover Flowchart

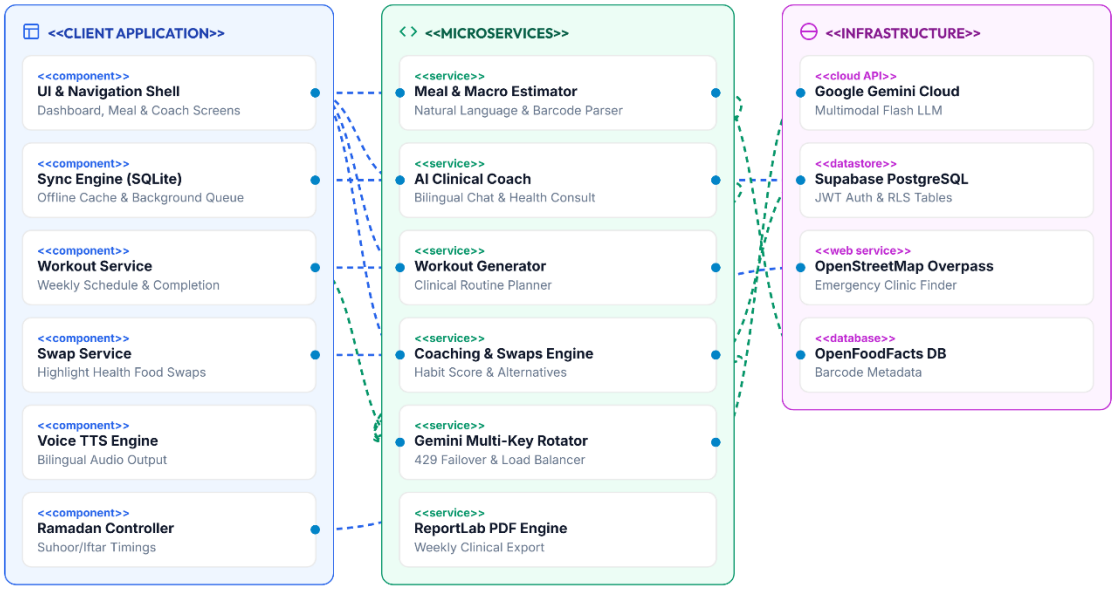
Figure 05 : Gemini Multi-Key Failover Flowchart



- **Quota Rate-Limit Monitoring:** The server monitors outbound Gemini API requests and intercepts rate-limiting exceptions (HTTP 429 Error).
- **Automated Key Interception:** When a rate-limit error is caught, the Uvicorn gateway blocks standard flow and triggers the thread-safe `GeminiPool` key rotation.
- **Redacted Key Switching:** The pool rotates the pointer to the next valid key index, automatically redacting and masking API keys inside the backend console logs.
- **Transparent Client Retries:** The API server transparently resends the inference request using the new rotated key, completing the transaction in less than 500ms.
- **High Availability:** The round-robin key failover pool ensures uninterrupted clinical chat, macro estimation, and workout planning with zero user-facing downtime.

3.6 Software Component Diagram

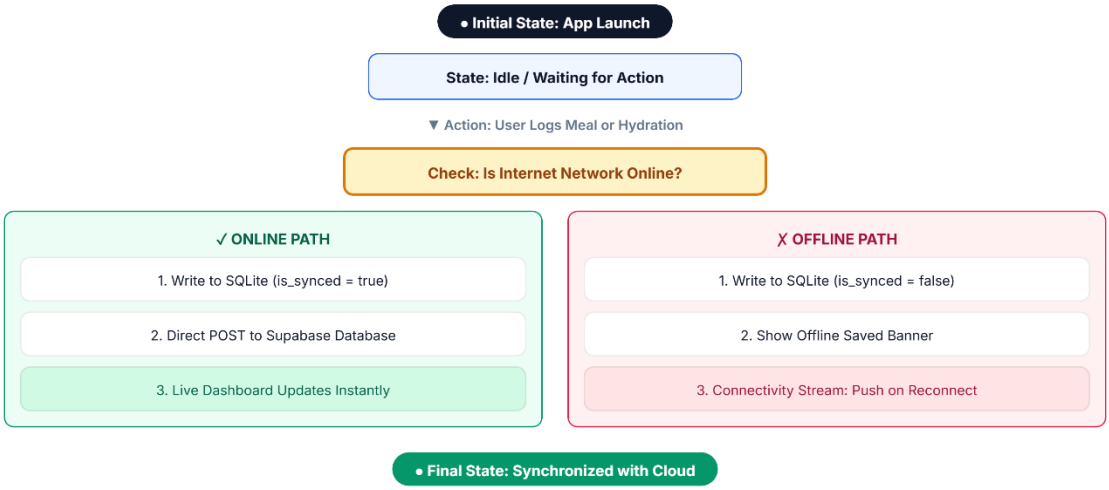
Figure 06 : Software Component Diagram



- **Modular Client Subsystems:** The frontend Flutter application separates UI modules, Local Caching Services, state Controllers, and the Speech TTS Engine.
- **Stateless Backend Endpoints:** The FastAPI backend acts as a stateless coordinator using distinct routers for auth, meal logging, clinical coaching, and PDF reports.
- **External Integration Connectors:** Unified interfaces decouple the core backend code from third-party services like OpenStreetMap geocoding and the OpenFoodFacts database.
- **Database Bridges:** SQLite database services (client-side) and Supabase Client APIs (cloud-side) are encapsulated to allow backend or storage modifications without UI changes.
- **Integration Interface Contracts:** Component adapters enforce strict JSON contracts for all communication, making backend testing and upgrades straightforward.

3.7 State Machine: Offline-First Synchronization

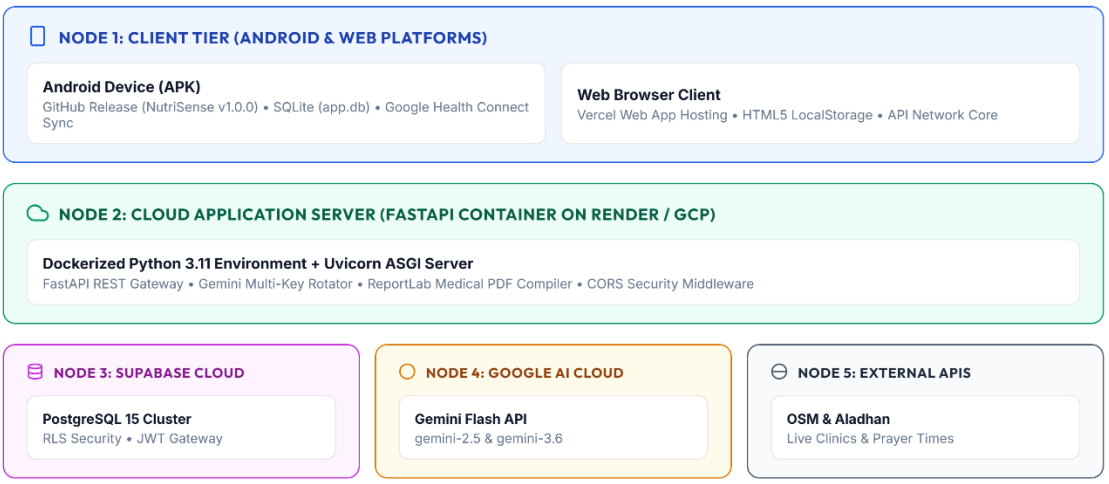
Figure 07 : State Machine: Offline-First Synchronization



- **Initial Sync State:** The application starts in an Idle state, actively listening to local inputs and monitoring network status streams.
- **Event Capture:** When a user logs meals, water, or workouts, the app checks the current network state before deciding the data routing path.
- **Resilient Offline Path:** If offline, the transaction is logged to SQLite with `is_synced = false`, and a visual "saved offline" message is displayed.
- **Auto-Reconnect Sync:** A connectivity stream listener detects when the network is restored and immediately triggers a background worker thread.
- **Supabase Push:** The worker batch-uploads pending records, handles synchronization conflicts, and updates local rows as fully synchronized (`is_synced = true`).

3.8 Physical & Cloud Deployment Diagram

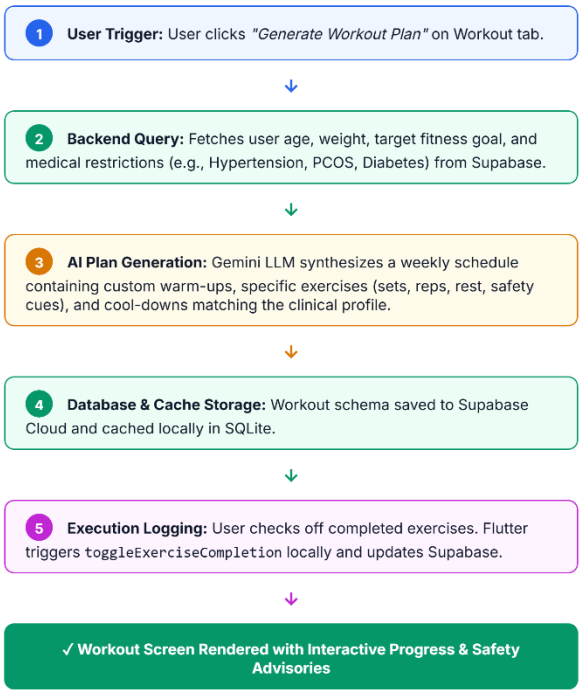
Figure 08 : Physical & Cloud Deployment Diagram



- **Multi-Platform Client Runtimes:** Clients run natively on Android devices (downloaded via GitHub Releases v1.0.0) or in modern web browsers (hosted on Vercel).
- **Containerized REST Nodes:** The FastAPI server runs in a containerized environment (hosted on Render / GCP) with Uvicorn ASGI servers handling client requests.
- **Database Cloud Layer:** Supabase runs on a managed cloud container hosting PostgreSQL, executing Row-Level Security policies and secure JSON Web Token verification.
- **Protected Server Gateways:** Cloudflare SSL edges secure the server nodes, shielding Uvicorn ASGI backends from DDoS attacks and unauthorized network requests.
- **AI Cloud Endpoints:** Outbound API gateways handle secure connections to the Google GenAI Cloud for high-throughput Gemini model executions.

3.9 AI Workout Plan Sequence Diagram

Figure 09 : AI Workout Plan Sequence Diagram



- **Interactive Request Event:** The user clicks the plan generation button in the app, initiating an asynchronous request to the backend `/workout/plan` endpoint.
- **Medical & Target Profile Lookup:** The FastAPI server queries Supabase to retrieve age, weight, target goal, and active medical restrictions (e.g., Hypertension or Diabetes).
- **Clinical Routine Synthesis:** The Gemini model parses user targets and restrictions to generate a balanced 7-day schedule with safety alerts and warm-ups.
- **Double-Database Caching:** The generated workout plan is saved in Supabase PostgreSQL and cached locally in the client SQLite database.
- **Execution Checklist State:** The user reviews exercises on the Workout tab and checks them off, syncing completion progress instantly between local and cloud databases.

- **Plan Generation Lifecycle:**

(1) User requests a plan.

(2) Backend queries profile metrics and medical restrictions from Supabase.

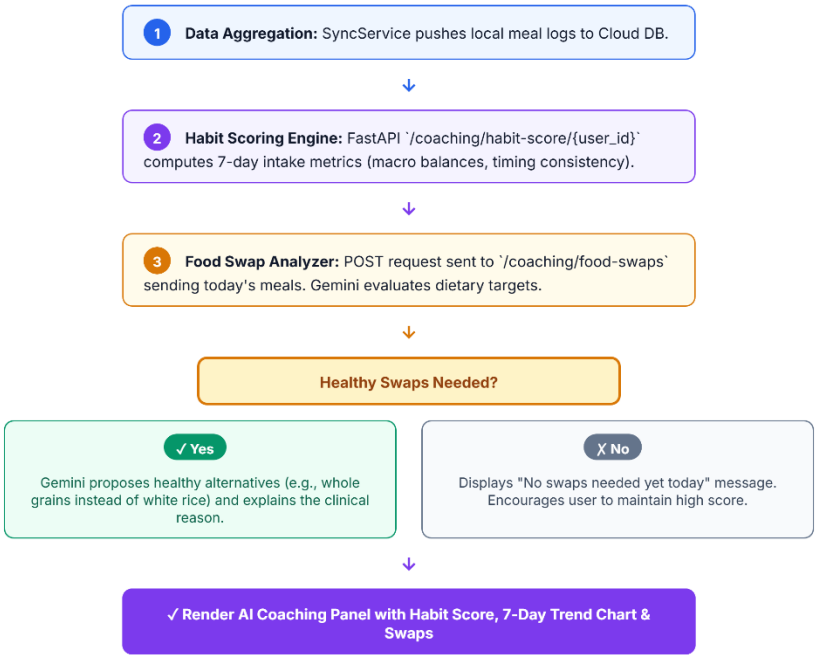
(3) Gemini AI synthesizes a weekly schedule containing custom warm-ups, specific exercises (sets, reps, rest, safety cues), and cool-downs matching the clinical profile.

(4) Workout plan is saved to Supabase and cached locally in SQLite.

(5) User checks off completed exercises, triggering toggleExerciseCompletion locally and updating Supabase.

3.10 Predictive AI Coaching & Recommended Swaps Data Flow

Figure 10 : Predictive AI Coaching & Recommended Swaps Data Flow



- **Nutrition Telemetry Sync:** The SyncService uploads all meal logs to Supabase, making the raw meal history available for rolling telemetry calculations.
- **Habit Ring Calculation:** FastAPI `/coaching/habit-score` processes 7-day intake data to calculate calorie consistency, macro balances, and logging streaks.
- **Gemini Food Swap Inspection:** Today's meal logs are analyzed by Gemini to search for high-glycemic or unhealthy items that conflict with the user's goals.
- **Clinical Swap Generation:** If violations are found, Gemini generates healthy South Asian swaps (e.g., brown rice for white rice) along with detailed clinical justifications.
- **Dashboard Render:** The frontend UI receives the scores and swap cards, updating habit rings, trend charts, and healthy swap recommendations in the Coaching panel.
- **Optimization Processing Loop:**

(1) SyncService pushes meal logs to Cloud DB.

(2) Habit Scoring Engine calculates 7-day habit scores.

(3) Food Swap Analyzer queries Gemini for today's diet swaps.

(4) If swaps are needed, Gemini proposes healthy alternatives and clinical reasons.

(5) AI Coaching Panel renders in UI with habit score, trend chart, and swaps.

4. Core Functional Module Specifications

4.1 Module 1: Natural Language AI Auto-Macro Estimator

The AI Auto-Macro Estimator eliminates the complexity of calorie counting. Users describe their meals in natural conversational language in English or Urdu without typing numbers.

- **Bilingual Natural Language Parsing:** Accurately understands colloquial meal names (e.g., '2 Aloo Parathas and 1 Chai', 'Daal Chawal with salad', 'Grilled Salmon with quinoa').
- **Portion Intelligence:** Automatically determines standard serving sizes when quantities are omitted, or scales macros dynamically based on specified quantities.
- **Regional & Desi Cuisine Grounding:** Built on an extensive South Asian nutritional knowledge matrix accounting for ghee, oil, and traditional cooking methods.

4.2 Module 2: High-Availability Multi-Key Gemini Failover Pool

To deliver enterprise SLA standards, the `GeminiPool` singleton implements thread-safe key management:

- **Dynamic Round-Robin Rotation:** Distributes API load across multiple Gemini API keys to balance quota consumption.
- **Instant HTTP 429 Interception:** Intercepts rate-limiting responses and switches keys in <50ms.
- **Security Redaction:** Automatically masks API key strings in server logs to prevent credential leakage.

4.3 Module 3: Multi-Member Family & Dependent Health Profiles

NutriSense allows a primary account holder to manage the nutritional wellness of their entire household:

- **Dependent Profiles**: Supports adding children, elderly parents, spouses, and siblings with custom age, gender, and calorie targets.
- **Allergy & Medical Rules per Member:** Enforces isolated safety guardrails (e.g., flagging peanut allergens for Child A while tracking low-sodium for Parent B).
- **1-Tap Dashboard Context Switching:** Allows switching the primary dashboard to visualize meals and calorie progress for any family member instantly.

4.4 Module 4: Ramadan Fasting Intelligence & Hydration Engine

A dedicated operational mode tailoring the entire app experience for religious fasting observances:

- **Solar Calculation Engine:** Automatically fetches astronomical Sehri (Fajr) and Iftar (Maghrib) timings based on exact GPS coordinates.
- **Live Fasting Countdown:** Displays hours and minutes remaining until Iftar / Sehri with an animated circular progress indicator.
- **Targeted Hydration Presets:** Provides 1-tap logging chips tailored for non-fasting windows (+500ml Iftar, +250ml Taraweeh, +500ml Sehri).
- **Dynamic Meal Categorization:** Replaces standard Breakfast/Lunch slots with Sehri, Iftar, and Post-Taraweeh Snack.

4.5 Module 5: Real-Time AI Clinical Chat & Bilingual Voice Coach

An interactive AI dietitian available 24/7 for conversational guidance, recipe suggestions, and health habit evaluation:

- **Rich Markdown Output:** Renders bold key terms, bulleted dietary plans, and numbered instructions cleanly.
- **Bilingual Text-To-Speech (TTS):** Reads coaching responses aloud in natural English or Urdu via `flutter_tts`.
- **Clinical Symptom Triaging:** Detects red-flag medical emergencies (chest pain, acute hypoglycemia, severe breathlessness) and directs users to immediate medical care.

4.6 Module 6: Emergency Clinic & Hospital Geo-Finder

Provides life-saving access to healthcare facilities during dietary or medical emergencies:

- **Overpass API Integration:** Real-time geospatial queries scan certified clinics, hospitals, and pharmacies within a 5km to 15km radius.
- **Facility Metadata:** Displays distance in kilometers, 24/7 emergency readiness flags, and direct phone contact links.
- **1-Tap Route Navigation:** Opens Google Maps or Apple Maps with pre-populated turn-by-turn emergency routing.

4.7 Module 7: Clinical Weekly Summary & PDF Receipt Generator

Aggregates 7-day health telemetry into professional clinical reports using ReportLab:

- **Caloric & Macronutrient Balance:** Visualizes average daily intake against prescribed medical targets.
- **Consistency & Habit Scoring:** Evaluates 30-day tracking consistency with streak metrics.
- **Doctor-Ready PDF Receipt:** Generates a downloadable clinical PDF summary suitable for sharing with physicians and dietitians.

4.8 Module 8: AI Workout Planner

Empowers users with personalized, clinically-safe physical training regimens tailored to their specific fitness goals, medical history, and physical profile:

- **Automated Weekly Schedule:** Generates a 7-day structured training protocol matching primary health goals (e.g., strength training for underweight, low-impact cardio for hypertension/diabetics).
- **Interactive Exercise Runtimes:** Tracks exercise execution and completion status in real-time, allowing users to toggle completion status on active screens.
- **Form Cues & Clinical Precautions:** Integrates precise training safety alerts and form cues generated dynamically by Gemini to prevent injuries.

4.9 Module 9: Predictive AI Coaching & Food Swaps

Provides proactive metabolic health optimization by analyzing rolling habits and delivering smart alternatives for high-glycemic or unhealthy logged items:

- **7-Day Consistency Habit Ring:** Computes a rolling consistency score based on dietary adherence, macronutrient balances, and logging streaks.
- **Gemini Food Swap Recommendations:** Automatically flags high-glycemic index foods or dietary violations and suggests healthy alternatives (e.g., swapping white rice for brown rice).
- **Clinical Rationale Explanations:** Accompanies every swap recommendation with a clear clinical reason detailing how the swap improves metabolic markers (e.g., reducing glycemic load).

5. Security, Compliance & Non-Functional Requirements

5.1 Authentication, Authorization & Session Security

- **JWT Bearer Tokens:** All client-server communications require cryptographically signed JSON Web Tokens issued by Supabase Auth.
- **Secure Local Storage:** Sensitive tokens and credentials are stored using hardware-backed platform keystores.
- **Safe Account Termination:** Comprehensive account deletion routines purge all relational database rows upon verified user confirmation.

5.2 Data Privacy & PostgreSQL Row-Level Security (RLS)

- **Row-Level Security Policies:** PostgreSQL RLS policies enforce that users can only SELECT, INSERT, UPDATE, or DELETE records where `user_id == auth.uid()`.
- **Family Access Isolation:** Dependent records and family meal logs are strictly restricted to the authenticated primary account holder.
- **Zero AI Data Retention:** Food scan queries and voice chats are processed ephemerally without using personal health records for public model training.

5.3 Performance, Scalability & Resource Optimization

- **Sub-600ms AI Response Latency:** Asynchronous FastAPI pipeline and optimized prompt tokens deliver instantaneous macro estimates.
- **60 FPS Smooth Rendering:** Flutter UI utilizes const constructors, isolated ListenableBuilder rebuild scopes, and image caching.
- **Low Power & Data Overhead:** Background sync operations execute only upon network state change or batch queue triggers.

5.4 Offline Resilience & Idempotent Sync Integrity

- **Zero Data Loss:** Every meal, water, and habit log is committed immediately to SQLite before initiating network transmission.
- **Idempotent Synchronization:** Background sync operations utilize unique UUID keys to prevent duplicate records upon reconnecting.